HW 6: Problem 1.1 Analysis

Problem 1.1: Strategic Network Formation

Part 1: Undirected Graph with Indicator Payoffs (Linear Model)

In the undirected model with indicator payoffs (referred to as the "Linear Model"), both participants of an edge pay the cost c because an edge $\{i, j\}$ increases both $|N_i|$ and $|N_j|$. The benefit of reaching another node does not decay with distance; it is simply 1 if a path exists. As seen in the code listing in Appendix A, we simulate these conditions starting from four distinct initial graph topologies: Empty, Complete, Star, and Cycle graphs.

- **Regime $c < 1$:** We simulated $c = 0.5$. The equilibrium reached from all initial states is a tree (8 edges for 9 nodes), which perfectly connects all nodes without any redundant edges. There does not appear to be a *unique* equilibrium graph topology (any spanning tree is an equilibrium because removing an edge breaks the component and losing ≥ 1 reachability is worse than saving $c = 0.5$, while adding an edge yields 0 additional reachability for cost c). The resulting trees are highly efficient since they maximize reachability at the minimum possible total edge cost. The total social welfare is mathematically $\Pi(\text{Tree}) = \sum_i (n-1) - 2c \cdot |E| = n(n-1) - 2c(n-1)$. Since any spanning tree (including a star graph) has exactly $|E| = n - 1$ edges and $\sum_i (n-1)$ reachability, they all yield the exact same mathematically optimal social welfare of $72 - 8 = 64$. A star is functionally equivalent to a line or any other tree in this specific non-decaying indicator payoff model.

- **Regime $1 < c < n - 1$:** We simulated $c = 4.0$. From all starting configurations (Empty, Complete, Star, Cycle), the simulation converges to the *empty graph* (0 edges). This is a unique equilibrium because any terminal node (a node with degree 1) pays $c > 1$ to maintain its single edge, but only gains a direct benefit of 1 from connecting to its neighbor. Even if that neighbor provides transitive reachability to the rest of the network, the terminal node's net marginal benefit for that edge is at most $1 + (n - 2) = n - 1$. However, the hub node connecting it must also pay $c > 1$ for that specific edge. Since the hub node is already connected to the rest of the network, that specific edge to the terminal node only provides a marginal reachability benefit of exactly 1. Since $c > 1$, the hub node will strictly prefer to sever the tie ($\Delta\text{Payoff} = c - 1 > 0$), causing the network to systematically unravel from the leaves inward until 0 edges remain. It is severely inefficient. For instance, a centralized star topology would have a socially optimal net positive welfare of $\Pi(\text{Star}) = n(n-1) - 2c(n-1) = 72 - 64 = 8$, but the empty graph leaves us at $\Pi(\text{Empty}) = 0$.

- **Regime $c > n - 1$:** We simulated $c = 10.0$. The unique equilibrium is the *empty graph*. Because $c > 8$, the maximum possible benefit of playing an edge (reaching all 8 other nodes) is still strictly less than the cost of a single edge ($c = 10$). The empty network is efficient in this regime because the maximum conceivable system-wide benefit of an edge (a cycle yielding $2(n-1)$ to the participants and perhaps some transitive benefit to others) is strictly outweighed by the direct costs. Formally, for any network G, $\Pi(G) \leq n(n-1) - 2c \cdot |E| < 0$ since $c > n-1$ means $2c > 2(n-1) \geq n$ (for $n > 2$). Thus $\Pi(\text{Empty}) = 0$ is optimal.

As shown in Figure 1, the equilibrium topography efficiently links every node.

Part 2: Directed Graph with Indicator Payoffs (Linear Model)

In the directed setting, nodes only pay for their outgoing edges, resolving the bilateral cost-sharing resistance seen in the undirected case. However, unlike undirected edges where a single link $\{A, B\}$ provides mutual two-way access, a directed edge (A, B) only provides A access to B, not vice-versa. This implies that a tree topology ($n - 1$ edges), which perfectly connects an undirected graph, cannot provide full reachability in a directed graph because there is no path back to the roots. The minimal structure to achieve full global reachability (strong connectivity) is a directed cycle (n edges).

- **Regime $c < 1$:** We simulated $c = 0.5$. Because $c < 1$, any unreached node provides 1 benefit for only c cost, prompting nodes to form links until the graph is strongly connected. A perfect directed cycle (9 edges for 9 nodes) is an efficient equilibrium with a maximal social welfare of $\Pi(\text{Cycle}) = \sum_i (n - 1 - c \cdot 1) = n(n - 1 - c) = 9(8 - 0.5) = 67.5$. The cycle is the most efficient topology because it provides every node full indicator reachability (+8) using the absolute mathematical minimum number of directed edges ($n = 9$). This shifts the optimal structure from an $n - 1$ edge tree in Part 1 to an n edge cycle in Part 2, a direct consequence of the one-way nature of directed traversal. While

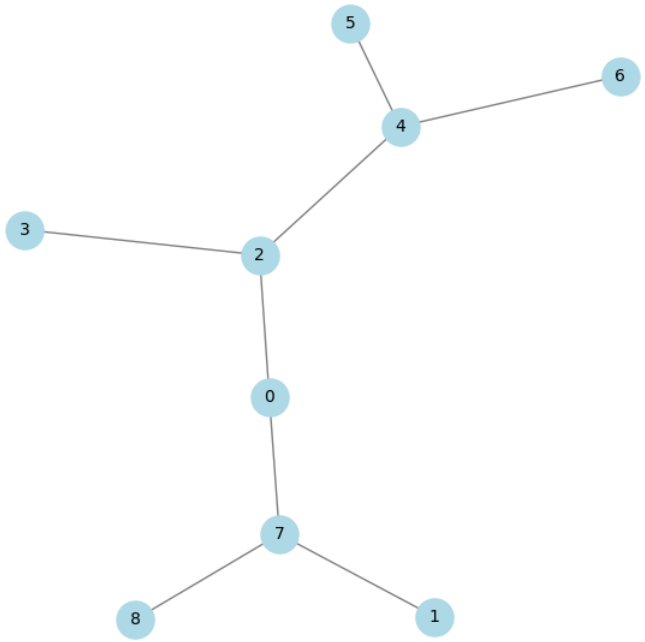

Figure 1: Equilibrium for Undirected Linear ($c < 1$). The network efficiently connects all nodes via a tree without any redundant edges.

the simulation sometimes converged to local Nash Equilibria with slightly more non-redundant edges depending on initialization, the cyclical motif remains paramount for reachability.

- **Regime $1 < c < n - 1$:** We simulated $c = 4.0$. The equilibria are deeply dependent on the initialization. If started from an empty graph or an inward star, no node wants to pay $c = 4.0$ to reach just 1 node, so the *empty graph* remains a stable equilibrium (inefficient, $\Pi = 0$). Conversely, if initialized as a complete graph, outward star, or cycle, the network trims down precisely to a *directed cycle* (9 edges). A cycle is a Nash Equilibrium because each node pays $c = 4.0$ for 1 outgoing edge but achieves a reachability of $8 \times 1 = 8$, netting +4.0. The cycle is an efficient topology here, achieving the maximum welfare $\Pi = n(n - 1 - c) = 36$.
- **Regime $c > n - 1$:** We simulated $c = 10.0$. As in the undirected case, the cost of a single edge exceeds the absolute maximum reachability benefit of $n - 1 = 8$. Thus, all nodes rapidly sever all outgoing edges, leaving the *empty graph* as the unique and efficient equilibrium ($\Pi = 0$).

Figure 2 visualizes this cyclical stability.

Part 3: Undirected Graph with Distance Payoffs (Connections Model)

In the connections model, the benefit explicitly decays with distance ($\delta = 0.5$).

- **Regime $c < \delta - \delta^2$:** We tested $c = 0.1$. As seen in Figure 3, the simulation instantly converges from any initialization to the *complete graph* (36 edges). Since $c < 0.25$, any node prefers adding a direct edge to another node rather than having an indirect path of distance 2 or more, as the gain ($\delta - \delta^2 = 0.25$) exceeds the cost c. Thus, it is the unique equilibrium and is efficient. The welfare mathematically is $\Pi(\text{Complete}) = n(n - 1)(\delta - c) = 9(8)(0.5 - 0.1) = 28.8$.
- **Regime $\delta - \delta^2 < c < \delta + (n/2 - 1)\delta^2$:** We tested $c = 0.8$. Starting from an empty graph (where no nodes share any edges), the simulation trivially terminates because adding the very first edge costs 0.8 but only provides a direct marginal benefit of $\delta = 0.5$. This empty equilibrium is rigorously inefficient ($\Pi = 0$).

  However, if we initialize the simulation from dense, highly-connected topologies (such as the Complete graph, Star graph, or Cycle graph), the simulation enters continuous cyclic behaviors (plotted in Figure 4). This instability occurs because nodes now disagree on the network's value depending on their algorithmic structural position. Peripheral nodes strictly benefit from finding cheap, short paths through central "hubs", so they aggressively desire to form or maintain links. But the central hub itself pays immense, compounding link costs (-0.8 per edge) without receiving equivalent distance payoffs (since it already had short paths to most nodes). As a result, the hub inevitably severs its ties to save

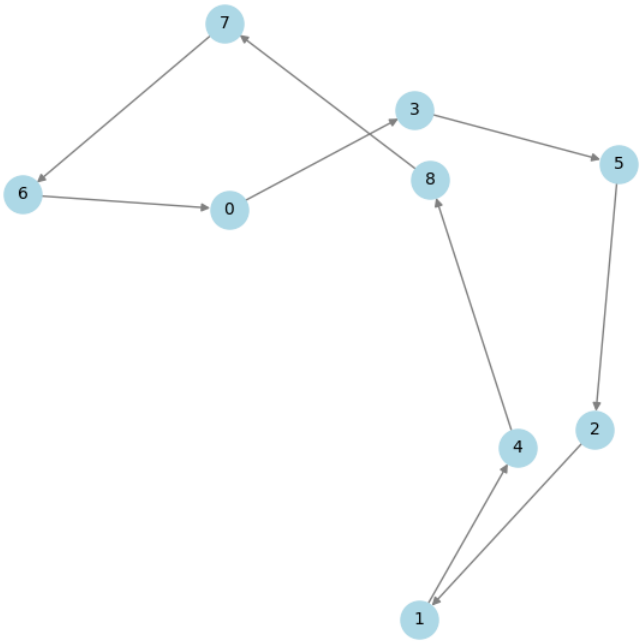

Figure 2: Equilibrium for Directed Linear ($1 < c < n - 1$) starting from a dense graph. The result trims precisely to an efficient directed cycle.

cost. This fractures the network, raising distances for the peripheral nodes, who then frantically try to build new edges to reconnect—causing the cycle to endlessly repeat without finding a stable Nash Equilibrium. This regime highlights a stark conflict between Nash equilibrium stability and social welfare: the socially optimum topology (the Star graph) has a strongly positive efficiency ($\Pi = 8(2(0.5) - 2(0.8)) + 56(0.5^2) = -4.8 + 14 = 9.2$), but no individual hub node will stably shoulder the personal cost to maintain it. Consequently, the network is either trapped at the 0 efficiency empty state or forced to endlessly cycle through suboptimal, transient inefficiencies.

- **Regime $c > \delta + (n/2 - 1)\delta^2$:** We tested $c = 2.0$. It firmly resolves back to the *empty graph* from almost all common starting points. Edge costs systematically overpower proximity rewards on all structural vectors ($2.0 > 0.5 + (3.5) * 0.25 = 1.375$), so nodes aggressively drop edges. The empty graph is uniquely efficient $\Pi = 0$ and acts as the predominant strong stable equilibrium here.

Figure 3 shows the uniform clustering of edges when direct distances are heavily rewarded, while Figure 4 demonstrates the cyclic instability in the intermediate cost regime.

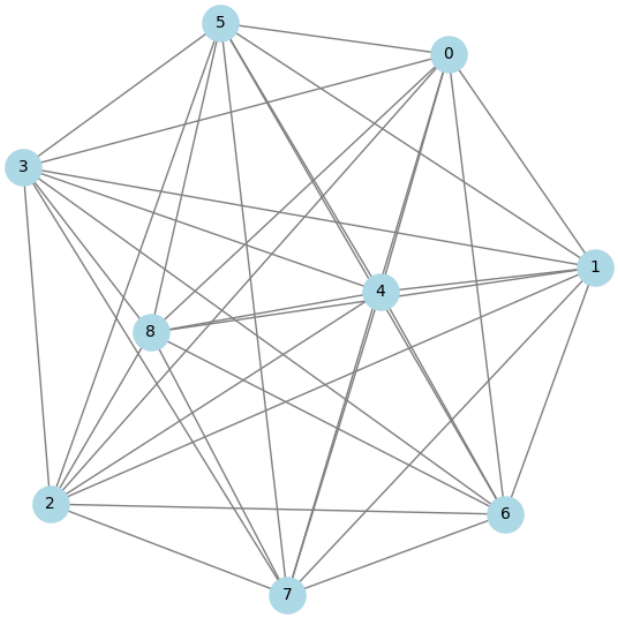

Figure 3: Equilibrium for Undirected Connections ($c < \delta - \delta^2$). Penalizing indirect paths quickly settles it into a complete graph.

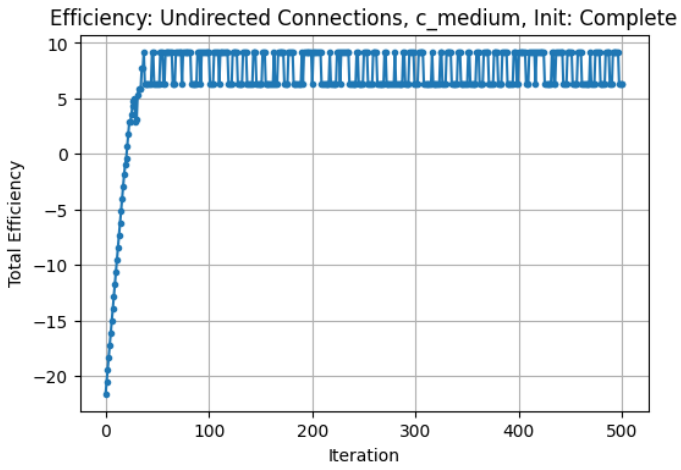

Figure 4: Efficiency convergence for Undirected Connections ($c = 0.8$) initialized from Complete graph. The system cycles continuously without a stable equilibrium.

Problem 1.2: Escaping the Cursed Maze

1. Why BFS/DFS Fails

Standard graph search algorithms like Depth-First Search (DFS) or Breadth-First Search (BFS) rely on a deterministic transition framework: if a node attempts to move North, it implicitly trusts that the resulting state will strictly be the room to the North. However, the cursed maze introduces a $p = 0.20$ probability of "slipping" on a conveyor belt, forcing the agent into a random adjacent room. This stochastic transition matrix destroys the absolute viability of static paths. Even if BFS finds the absolute shortest sequence of cardinal moves to an exit, it cannot guarantee that sequence will actually be executed. Crucially, algorithms like BFS fail to internalize risk: a shortest path that skirts tightly along the edge of multiple snake rooms is astronomically dangerous under a 20% slip rate, but BFS cannot distinguish this probabilistic danger from a safe open corridor as long as the step count is identical.

2. TD(0) Learning and Escaping

To successfully navigate the maze, we modeled the problem as Reinforcement Learning over a Markov Decision Process, specifically utilizing a Temporal Difference TD(0) value update rule. We learned the state value function $V(s)$ by exploring the maze for 998 episodes using an ε-greedy policy to balance exploiting known safe paths and exploring unknown areas.

Hyperparameters Configured:

- **Learning Rate** $\alpha = 0.5$: A moderate learning rate to quickly backpropagate terminal signals while smoothing out the noise from stochastic collisions.
- **Exploration** ε: Decayed exponentially from 1.0 down to 0.05 over the 998 episodes using a 0.995 multiplier per episode, ensuring broad early exploration that gradually settles into confident exploitation.
- **Rewards**: Snakes were initialized and permanently fixed at -100.0, escapes fixed at $+100.0$. Standard, unexplored square rooms began at 0.0.

During our empirical test consisting of 1000 iterations with a purely greedy policy ($\varepsilon = 0$), the agent successfully escaped the maze **1000 out of 1000** times, achieving a 100% survival rate. Because the agent directly learned $V(s)$ from stochastic experience, it implicitly learned to heavily discount the values of rooms directly physically adjacent to snakes—punishing the areas where slipping would be lethal. This phenomenon is directly supported by Figure 5, which shows distinct low-value suppression zones surrounded by the 'S' rooms. Thus, the greedy policy naturally prefers longer, safer corridors over dangerous shortcuts.

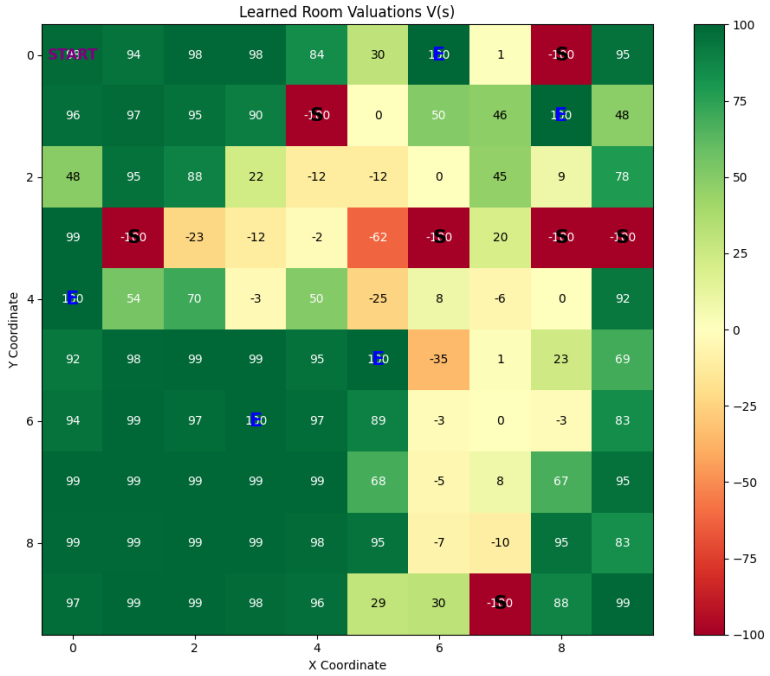

Figure 5: The converged $V(s)$ state-value grid after 998 episodes. Safest, most attractive rooms radiate outward from the escapes (E), while regions bordering snakes (S) are heavily suppressed.

The fully converged state-valuation grid $V(s)$ that drives the optimal policy is plotted in Figure 5. The complete Python implementation for learning and solving the maze (`solve_maze.py`) is provided in Appendix A.

Problem 2: Theory

2.1 Routing Games with Tolls

We are given a routing game with N players, each with traffic $r_i = 1$. The original link cost functions are $c_e(\cdot)$ and the social cost is:

$$W(f) = \sum_{e \in E} f_e c_e(f_e).$$

The new tolled cost function is:

$$c_e^*(x) = c_e(x) + (x-1)\big(c_e(x) - c_e(x-1)\big).$$

We must prove that any f^* minimizing $W(f)$ is a pure Nash Equilibrium (NE) of the new game.

Step 1: Define the Rosenthal potential and show it is a valid potential function.

Define the Rosenthal potential for the new game:

$$\Phi^*(f) = \sum_{e \in E} \sum_{x=1}^{f_e} c_e^*(x).$$

We claim Φ^* is an *exact potential function*: when a single player i deviates from path P to path P', the change in Φ^* equals the change in that player's individual cost. To see this, consider player i switching from P to P'. For edges $e \in P \setminus P'$, the flow drops from f_e to $f_e - 1$, so:

$$\Delta_e \Phi^* = -c_e^*(f_e).$$

For edges $e \in P' \setminus P$, the flow rises from f_e to $f_e + 1$, so:

$$\Delta_e \Phi^* = +c_e^*(f_e + 1).$$

These are exactly the marginal costs player i saves and incurs, which equals the change in player i's total cost $\sum_{e \in P'} c_e^*(f_e + 1) - \sum_{e \in P} c_e^*(f_e)$. Therefore $\Delta\Phi^* = \Delta(\text{cost of player } i)$, confirming Φ^* is a valid potential function.

Step 2: Any minimum of Φ^* is a Nash Equilibrium, and one exists.

Since Φ^* is an exact potential, any unilateral deviation that improves a player's cost strictly decreases Φ^*. Equivalently, if f is a local minimum of Φ^*, no player can benefit from deviating—so f is a NE. Because the set of feasible integer flows is finite (each player has finitely many paths), Φ^* attains its global minimum. Therefore, a pure NE always exists.

Step 3: Show $\Phi^*(f) = W(f)$, so the minimizer of W is a NE.

We simplify the inner sum of Φ^* for a single edge e by substituting c_e^*:

$$\begin{aligned}\sum_{x=1}^{f_e} c_e^*(x) &= \sum_{x=1}^{f_e} \big[c_e(x) + (x-1)c_e(x) - (x-1)c_e(x-1)\big] \\ &= \sum_{x=1}^{f_e} \big[x\, c_e(x) - (x-1)c_e(x-1)\big].\end{aligned}$$

Setting $g_e(x) = x\, c_e(x)$, this is a telescoping sum:

$$\sum_{x=1}^{f_e} \big[g_e(x) - g_e(x-1)\big] = g_e(f_e) - g_e(0) = f_e c_e(f_e),$$

since $g_e(0) = 0$. Summing over all edges:

$$\Phi^*(f) = \sum_{e \in E} f_e c_e(f_e) = W(f).$$

Conclusion.

Since $\Phi^*(f) = W(f)$, the global minimizer f^* of the original social cost $W(f)$ is also the global minimizer of Φ^*. By Step 2, any global minimizer of Φ^* is a pure Nash Equilibrium of the tolled game. Therefore, f^* is a Nash Equilibrium. ■

2.2 Alternating Offers

Eric gives Anagha and Albert $3000 to split over at most three rounds. Offers must be non-negative integers. Each time an offer is *rejected*, the total pool shrinks by $1000. We solve by backward induction.

Part (a): Both players fully rational (maximizing own payoff).

Round 3 (Anagha's final offer, pool = $1000). If Albert rejects, both receive $0. Since Albert is rational, he accepts any offer $A_3 \geq 0$ (weakly better than $0). Anagha therefore offers Albert $A_3 = \$0$ and keeps $1000.

Round 3 outcome: Anagha $1000, Albert $0.

Round 2 (Albert's offer to Anagha, pool = $2000). Anagha knows she will receive $1000 if she rejects (Round 3 outcome). She accepts iff $A_2 \geq 1000$. Albert minimizes A_2 to keep the rest, so Albert offers $A_2 = \$1000$.

Round 2 outcome: Anagha $1000, Albert $1000.

Round 1 (Anagha's opening offer to Albert, pool = $3000). Albert knows he receives $1000 if he rejects (Round 2 outcome). He accepts iff $A_1 \geq 1000$. Anagha minimizes A_1, so she offers $A_1 = \$1000$, keeping $2000 for herself.

Answer (a): Anagha offers Albert $1000. Anagha keeps $2000.

Part (b): Albert is spiteful — he only accepts if he gets *strictly more* than Anagha, and maximizes his absolute advantage (his amount minus Anagha's). When indifferent, he rejects Anagha's offer.

Round 3 (Anagha's final offer, pool = $1000). If Albert rejects, both get $0 (advantage = 0). Albert accepts Anagha's offer of A_3 only if:

1. $A_3 > 1000 - A_3$ (strictly more than Anagha), i.e. $A_3 \geq 501$.
2. His advantage from accepting $2A_3 - 1000 > 0$, i.e. $A_3 \geq 501$.
3. He is not indifferent: since rejection gives advantage 0, he rejects whenever $2A_3 - 1000 = 0$, i.e. $A_3 = 500$ (here he does not get strictly more anyway, so this is covered).

Anagha wants to maximize her share $1000 - A_3$. To secure acceptance she must offer $A_3 = 501$, giving herself $499. Offering $A_3 \leq 500$ leads to rejection and $0 for Anagha. Since $499 > 0$, Anagha offers $501.

Round 3 outcome: Anagha $499, Albert $501 (advantage = +$2).

Round 2 (Albert's offer to Anagha, pool = $2000). Anagha accepts iff $A_2 \geq 499$ (her Round 3 payoff). Albert wants $2000 - A_2 > A_2$ (strictly more), i.e. $A_2 \leq 999$, and maximizes his advantage $2000 - 2A_2$ by minimizing A_2. The minimum acceptable to Anagha is $A_2 = 499$. Albert offers $A_2 = \$499$, keeping $1501 with advantage $1501 - 499 = \$1002$.

Round 2 outcome: Anagha $499, Albert $1501 (advantage = +$1002).

Round 1 (Anagha's opening offer to Albert, pool = $3000). Albert accepts iff:

1. $A_1 > 3000 - A_1$, i.e. $A_1 \geq 1501$.
2. His advantage from accepting $2A_1 - 3000$ strictly exceeds his advantage from rejecting $1002, i.e. $A_1 > 2001$, meaning $A_1 \geq 2002$ (integers).
3. He does not reject out of spite: at $A_1 = 2001$, advantage = 1002 = rejection advantage, so he rejects (indifference $\Rightarrow$ reject).

Anagha keeps $3000 - A_1$. Offering $A_1 = 2002$ yields $998 for herself, which is better than the $499 she would get if Albert rejects (Round 2 outcome). Any offer $A_1 \leq 2001$ is rejected, leaving Anagha with $499.

Since $998 > 499$, Anagha offers exactly $A_1 = \$2002$.

Answer (b): Anagha offers Albert $2002. Anagha keeps $998.

2.3 Infinite Equilibria

Answer: No, I do not agree with Ann. Ann's statement holds for $m \leq 2$, but fails for any $m \geq 3$. Thus, the maximum m such that her statement holds is $\mathbf{m = 2}$.

1. Why Ann's statement holds for $m = 2$ (Upper Bound Proof)

Consider a general 2×2 game where all 4 payoffs for a player are distinctly different. A generic game has at most 3 Nash Equilibria (up to 2 pure and 1 totally mixed). For there to be an infinite number of equilibria, there must be a *continuous family* of mixed-strategy equilibria. This continuous family implies at least one player must be willing to play a continuous range of probability mixtures, which means the *other* player must be totally indifferent between their actions for that entire range of mixtures.

Let Column's mixture be $(q, 1-q)$. Consider the generic 2×2 payoff matrix for Row:

	L	R
A	$a, \cdot$	$b, \cdot$
B	$c, \cdot$	$d, \cdot$

where a, b, c, d are all distinct (Ann's assumption). For Row to be indifferent between A and B, it must be that:

$$q \cdot u_r(A, L) + (1-q) \cdot u_r(A, R) = q \cdot u_r(B, L) + (1-q) \cdot u_r(B, R)$$

Rearranging yields a linear equation in q:

$$q[u_r(A, L) - u_r(B, L) - u_r(A, R) + u_r(B, R)] = u_r(B, R) - u_r(A, R)$$

Since all payoffs for Row are distinct, we strictly have $u_r(B, R) \neq u_r(A, R)$, meaning the right-hand side is non-zero. The only way this equation holds for a continuous range of q is if both the coefficient of q and the right-hand side are strictly 0 (which is impossible as proven). Thus, this equation has **at most one** root $q^* \in (0, 1)$.

Therefore, Column must play exactly q^* for Row to mix, and similarly Row must play exactly p^* for Column to mix. Because there is exactly one probability distribution that makes the other indifferent, there is at most one totally mixed Nash equilibrium (p^*, q^*). Since both pure and mixed equilibria are finite, the total number of Nash equilibria in a 2×2 game with distinct payoffs is finite. Ann's statement holds for $m = 2$.

2. Counterexample for $\mathbf{m = 3}$

Consider the following 3×3 payoff matrix:

	L	C	R
T	$1, 10$	$11, 2$	$6, 6$
M	$12, 0$	$0, 8$	$4, 4$
B	$2, 1$	$3, 3$	$5, 5$

Notice that all 9 payoffs for Row $(1, 11, 6, 12, 0, 4, 2, 3, 5)$ are strictly distinct. All 9 payoffs for Column $(10, 2, 6, 0, 8, 4, 1, 3, 5)$ are also strictly distinct. Ann's assumption is perfectly satisfied.

Suppose Row plays the mixed strategy $p = (0.5, 0.5, 0)$. Let us calculate Column's expected payoffs:

- $E_c[L] = 0.5(10) + 0.5(0) = 5$
- $E_c[C] = 0.5(2) + 0.5(8) = 5$
- $E_c[R] = 0.5(6) + 0.5(4) = 5$

Column is perfectly indifferent among all three actions! Therefore, Column can play *any* mixed strategy $q = (q_1, q_2, q_3)$ without losing utility.

Next, we need Column to play a mixed strategy that makes Row indifferent between T and M while B remains strictly worse as a best response. Let Column play a mixture q:

$$E_r[T] = q_1 + 11q_2 + 6q_3$$
$$E_r[M] = 12q_1 + 0q_2 + 4q_3$$

Setting $E_r[T] = E_r[M]$ and substituting $q_3 = 1 - q_1 - q_2$ yields:

$$6 - 5q_1 + 5q_2 = 4 + 8q_1 - 4q_2 \implies \mathbf{13q_1 - 9q_2 = 2}$$

Because this equality defines a line segment in the standard simplex space, there are an ***infinite*** number of valid (q_1, q_2, q_3) distributions. For example, if we pick $q_1 = 4/13 \approx 0.307$, we get $q_2 = 2/9 \approx 0.222$ and $q_3 \approx 0.470$. At this distribution point, Row's expected payoffs are:

- $E_r[T] = E_r[M] = \frac{652}{117} \approx \mathbf{5.57}$
- $E_r[B] = 2(\frac{36}{117}) + 3(\frac{26}{117}) + 5(\frac{55}{117}) = \frac{425}{117} \approx \mathbf{3.63}$

Since $E_r[T] = E_r[M] > E_r[B]$ strictly, playing $(0.5, 0.5, 0)$ is a valid best response for Row.

Thus, any strategy profile where Row plays $(0.5, 0.5, 0)$ and Column plays $(q_1, q_2, 1 - q_1 - q_2)$ satisfying $13q_1 - 9q_2 = 2$ corresponds to a Nash Equilibrium. Since there are uncountably infinite points on that line segment, there are infinite equilibria.

2.4 PoA Analysis for Generalized Second Price Auction

Part (a): The pure PoA can be arbitrarily large ($n = 2$).

For any $r > 1$, set $n = 2$ with the following parameters:

- Valuations: $v_1 = r$, $v_2 = 1$ (so advertiser 1 is higher-value).
- Clicks: $\alpha_1 = 1$, $\alpha_2 = 0$ (slot 2 gives no clicks).
- Bids: $b_1 = 0$, $b_2 = r + 1$.

This is a pure NE:

- Advertiser 2 wins slot 1, pays $b_1 = 0$, earning utility $\alpha_1(v_2 - 0) = 1 > 0$. Deviating to slot 2 yields $\alpha_2 v_2 = 0 < 1$. No incentive to deviate.
- Advertiser 1 is in slot 2 with utility $\alpha_2 v_1 = 0$. Deviating to slot 1 requires outbidding $b_2 = r+1 > v_1 = r$, giving utility $\alpha_1(v_1 - b_2) = r - (r + 1) = -1 < 0$. No incentive to deviate.

The optimal allocation assigns advertiser 1 to slot 1:

$$W(b^*) = \alpha_1 v_1 + \alpha_2 v_2 = r, \qquad W(\text{NE}) = \alpha_1 v_2 + \alpha_2 v_1 = 1.$$

$$\text{PPoA} = \frac{r}{1} = r. \qquad \blacksquare$$

Part (b)(i): Dominated strategy in the example.

In the above NE, advertiser 2 bids $b_2 = r + 1 > v_2 = 1$, a non-conservative bid. This is a *dominated strategy.* The conservative bid $b_2' = v_2 = 1$ weakly dominates b_2:

- If $b_1 < v_2 = 1$: advertiser 2 wins slot 1 with either bid, paying b_1 either way. Utilities are identical.
- If $b_1 \in (v_2, b_2) = (1, r + 1)$: with $b_2 = r + 1$, advertiser 2 still wins slot 1 but pays $b_1 > v_2$, getting utility $v_2 - b_1 < 0$. With $b_2' = 1 < b_1$, advertiser 2 drops to slot 2, paying 0, getting utility $\alpha_2 v_2 = 0 > v_2 - b_1$. The conservative bid is *strictly* better.
- If $b_1 \geq b_2$: advertiser 2 is in slot 2 in both cases. Same utility.

The **risk** of overbidding: if an opponent bids between v_2 and b_2, advertiser 2 wins a slot but pays more per click than the click is worth, guaranteeing negative utility. The conservative bid avoids this exposure entirely.

Part (b)(ii): Non-conservative bids are always dominated ($n \geq 2$).

Claim: For any advertiser i with $b_i > v_i$, the bid $b_i' = v_i$ weakly dominates b_i.

Fix all other bids b_{-i}. Let j be the slot advertiser i wins with b_i, and j' with b_i'. Since $b_i' < b_i$, we have $j \leq j'$ (weakly better or same slot with the higher bid).

- **Same slot** ($j = j'$): The price paid per click is determined solely by the next-highest bid, which is unchanged. Utilities are equal.
- **Strictly better slot with** b_i ($j < j'$): Advertiser i gained slot j by outbidding some advertiser k whose bid b_k satisfies $v_i = b_i' \leq b_k < b_i$. Advertiser i then pays $b_k \geq v_i$ per click in slot j, earning utility $\alpha_j(v_i - b_k) \leq 0$. With $b_i' = v_i$, advertiser i occupies slot $j' > j$ and pays at most $b_k' < v_i$ per click (any bid below v_i that beat out), yielding utility $\alpha_{j'}(v_i - b_k') \geq 0$. The conservative bid is *strictly better* whenever this case occurs.

Thus $b_i' = v_i$ weakly dominates $b_i > v_i$, with a strict improvement whenever the higher bid "wins" a slot at a price above v_i. $\blacksquare$

Part (b)(iii): With $n = 2$ and both advertisers conservative, PPoA ≤ 1.25; this bound is tight.

Upper bound proof. Following the hint, scale so that $\alpha_1 = 1$, $\alpha_2 = r \leq 1$, $v_1 + r v_2 = 1$, $v_1 > v_2 > 0$. The optimal welfare is $W^* = 1$.

If the NE has the correct allocation (advertiser 1 in slot 1), $W(\text{NE}) = W^* = 1$, and the ratio is 1. Now suppose the NE has the *reversed* allocation (advertiser 2 bids $b_2 \leq v_2$ and wins slot 1; advertiser 1 bids $b_1 < b_2$ and is in slot 2). NE conditions require:

1. Advertiser 1 prefers slot 2: $\alpha_2 v_1 \geq \alpha_1(v_1 - b_2) \Rightarrow b_2 \geq v_1(1 - r)$.

2. Advertiser 2 prefers slot 1: $\alpha_1(v_2 - b_1) \geq \alpha_2 v_2 \Rightarrow b_1 \leq v_2(1-r)$.

3. Conservative bids: $b_2 \leq v_2$ (combined with condition 1: $v_1(1-r) \leq v_2$).

The welfare ratio in the reversed NE is:

$$\rho = \frac{W^*}{W(\text{NE})} = \frac{v_1 + rv_2}{v_2 + rv_1}.$$

We claim $\rho \leq 5/4$. This is equivalent to $4(v_1 + rv_2) \leq 5(v_2 + rv_1)$, i.e., $v_1(4-5r) \leq v_2(5-4r)$.

- If $r \geq 4/5$: LHS $= v_1(4-5r) \leq 0$, while RHS $= v_2(5-4r) > 0$ (since $r < 5/4$). Done.
- If $r < 4/5$: We need $v_2/v_1 \geq (4-5r)/(5-4r)$. From condition 3 of the NE, $v_2 \geq v_1(1-r)$, so it suffices to check $(1-r) \geq (4-5r)/(5-4r)$, i.e., $(1-r)(5-4r) \geq 4-5r$:

$$5 - 9r + 4r^2 \geq 4 - 5r \iff 1 - 4r + 4r^2 \geq 0 \iff (1-2r)^2 \geq 0. \checkmark$$

Therefore $\rho \leq 5/4$ in all cases, so PPoA ≤ 1.25. ■

Tight example achieving PPoA $= 1.25$. Equality holds when $(1-2r)^2 = 0$, i.e., $r = \alpha_2/\alpha_1 = 1/2$, and $v_2/v_1 = 1 - r = 1/2$. Set:

$$\alpha_1 = 1, \quad \alpha_2 = \tfrac{1}{2}, \quad v_1 = 2, \quad v_2 = 1, \quad b_1 = 0, \quad b_2 = 1.$$

Advertiser 2 wins slot 1 (bids $b_2 = 1 > b_1 = 0$), pays $b_1 = 0$ per click:

- Advertiser 2: utility $= \alpha_1(v_2 - 0) = 1$. Dropping to slot 2 gives $\alpha_2 v_2 = 0.5 < 1$. No deviation.
- Advertiser 1: utility $= \alpha_2 v_1 = 1$. Raising bid to win slot 1 pays $b_2 = 1 = v_2$, giving $\alpha_1(v_1 - 1) = 1$. Indifferent (no strict improvement).

This is a valid conservative pure NE ($b_2 = v_2$, $b_1 = 0 \leq v_1$). The welfare:

$$W(\text{NE}) = \alpha_1 v_2 + \alpha_2 v_1 = 1 + 1 = 2, \qquad W^* = \alpha_1 v_1 + \alpha_2 v_2 = 2 + 0.5 = 2.5.$$

$$\text{PPoA} = \frac{2.5}{2} = 1.25. \qquad \blacksquare$$

A Code Listing

```python
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

def linear_payoffs(G, c, graph_type='undirected'):
    n = G.number_of_nodes()
    payoffs = np.zeros(n)
    for i in range(n):
        if graph_type == 'undirected':
            ni = G.degree(i)
            reach = len(nx.node_connected_component(G, i)) - 1
        else:
            ni = G.out_degree(i)
            reach = len(nx.descendants(G, i))

        payoffs[i] = -c * ni + reach
    return payoffs

def connections_payoffs(G, c, delta, graph_type='undirected'):
    n = G.number_of_nodes()
    payoffs = np.zeros(n)
    path_lengths = dict(nx.shortest_path_length(G))
    for i in range(n):
        if graph_type == 'undirected':
            ni = G.degree(i)
        else:
            ni = G.out_degree(i)

        benefit = 0
```

```
        for j in range(n):
            if i != j:
                if j in path_lengths[i]:
                    d = path_lengths[i][j]
                    benefit += delta ** d

        payoffs[i] = -c * ni + benefit
    return payoffs

def get_possible_actions(G, u, graph_type='undirected'):
    actions = [('none', None)]
    n = G.number_of_nodes()

    if graph_type == 'undirected':
        neighbors = set(G.neighbors(u))
    else:
        neighbors = set(G.successors(u))

    non_neighbors = set(range(n)) - neighbors - {u}

    for w in non_neighbors:
        actions.append(('add', w))

    for v in neighbors:
        actions.append(('remove', v))

    for v in neighbors:
        for w in non_neighbors:
            actions.append(('swap', (v, w)))

    return actions

def apply_action(G, u, action, graph_type='undirected'):
    G_new = G.copy()
    act_type, params = action

    if act_type == 'none':
        pass
    elif act_type == 'add':
        G_new.add_edge(u, params)
    elif act_type == 'remove':
        G_new.remove_edge(u, params)
    elif act_type == 'swap':
        v, w = params
        G_new.remove_edge(u, v)
        G_new.add_edge(u, w)

    return G_new

def run_simulation(init_G, c, model='linear', delta=0.5, graph_type='undirected', max_iters=500):
    G = init_G.copy()
    n = G.number_of_nodes()

    history = [tuple(sorted(G.edges()))]

    if model == 'linear':
        eff = sum(linear_payoffs(G, c, graph_type))
    else:
        eff = sum(connections_payoffs(G, c, delta, graph_type))
    efficiencies = [eff]

    for t in range(max_iters):
        u = t % n

        actions = get_possible_actions(G, u, graph_type)

        best_payoff = -float('inf')
        best_actions = []

        for action in actions:
            G_new = apply_action(G, u, action, graph_type)
            if model == 'linear':
                payoff = linear_payoffs(G_new, c, graph_type)[u]
            else:
                payoff = connections_payoffs(G_new, c, delta, graph_type)[u]

```

```
            if payoff > best_payoff + 1e-9:
                best_payoff = payoff
                best_actions = [action]
            elif abs(payoff - best_payoff) < 1e-9:
                best_actions.append(action)

        none_in_best = False
        for a in best_actions:
            if a[0] == 'none':
                none_in_best = True
                break

        if none_in_best:
            chosen_action = ('none', None)
        else:
            chosen_action = best_actions[np.random.choice(len(best_actions))]

        G = apply_action(G, u, chosen_action, graph_type)
        history.append(tuple(sorted(G.edges())))

        if model == 'linear':
            eff = sum(linear_payoffs(G, c, graph_type))
        else:
            eff = sum(connections_payoffs(G, c, delta, graph_type))
        efficiencies.append(eff)

        if t >= n:
            edges_same = True
            for i in range(1, n+1):
                if history[-1] != history[-(i+1)]:
                    edges_same = False
                    break
            if edges_same:
                break

    return G, history, efficiencies

def calculate_efficiency(G, c, model, delta, graph_type):
    if model == 'linear':
        payoffs = linear_payoffs(G, c, graph_type)
    else:
        payoffs = connections_payoffs(G, c, delta, graph_type)
    return sum(payoffs)

def generate_inits(n, graph_type):
    inits = {}
    if graph_type == 'undirected':
        inits['Empty'] = nx.empty_graph(n)
        inits['Complete'] = nx.complete_graph(n)
        inits['Star'] = nx.star_graph(n-1)
        inits['Cycle'] = nx.cycle_graph(n)
    else:
        inits['Empty'] = nx.empty_graph(n, create_using=nx.DiGraph)
        inits['Complete'] = nx.complete_graph(n, create_using=nx.DiGraph)
        star_out = nx.DiGraph()
        star_out.add_nodes_from(range(n))
        for i in range(1, n):
            star_out.add_edge(0, i)
        inits['Star_Out'] = star_out

        star_in = nx.DiGraph()
        star_in.add_nodes_from(range(n))
        for i in range(1, n):
            star_in.add_edge(i, 0)
        inits['Star_In'] = star_in

        cycle = nx.DiGraph()
        cycle.add_nodes_from(range(n))
        for i in range(n):
            cycle.add_edge(i, (i+1)%n)
        inits['Cycle'] = cycle
    return inits

def plot_graph(G, title, filename):
    plt.figure(figsize=(6,6))
    pos = nx.spring_layout(G, seed=42)
```

```
    nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=500, font_size=10,
            arrows=isinstance(G, nx.DiGraph), edge_color='gray')
    plt.title(title)
    plt.savefig(f'/Users/mhanisch/projects/psets/q2/144/hw6/latex/figs/{filename}')
    plt.close()

def plot_convergence(efficiencies, title, filename):
    plt.figure(figsize=(6,4))
    plt.plot(efficiencies, marker='o', markersize=3)
    plt.xlabel('Iteration')
    plt.ylabel('Total Efficiency')
    plt.title(title)
    plt.grid(True)
    plt.savefig(f'/Users/mhanisch/projects/psets/q2/144/hw6/latex/figs/{filename}', bbox_inches='tight')
    plt.close()

if __name__ == '__main__':
    n = 9
    out_lines = []

    # 1. Undirected Linear
    out_lines.append("=== Part 1: Undirected Linear ===")
    c_vals = [0.5, 4.0, 10.0]
    c_labels = ['c_lt_1', '1_lt_c_lt_n-1', 'c_gt_n-1']
    for c, label in zip(c_vals, c_labels):
        out_lines.append(f"\\nRegime: {label} (c = {c})")
        inits = generate_inits(n, 'undirected')
        for init_name, init_G in inits.items():
            G_eq, history, efficiencies = run_simulation(init_G, c, model='linear', graph_type='undirected')
            eff = calculate_efficiency(G_eq, c, 'linear', delta=0, graph_type='undirected')
            out_lines.append(f"  Init: {init_name:<10} | Steps: {len(history)-1:<4} | Efficiency: {eff:<6.2f} | Num Edges: {G_eq.number_of_edges()}")
            plot_graph(G_eq, f"Undirected Linear, {label}, Init: {init_name}", f"undirected_linear_{label}_{init_name}.png")
            plot_convergence(efficiencies, f"Efficiency: Undirected Linear, {label}, Init: {init_name}", f"conv_undirected_linear_{label}_{init_name}.png")

    # 2. Directed Linear
    out_lines.append("\\n=== Part 2: Directed Linear ===")
    for c, label in zip(c_vals, c_labels):
        out_lines.append(f"\\nRegime: {label} (c = {c})")
        inits = generate_inits(n, 'directed')
        for init_name, init_G in inits.items():
            G_eq, history, efficiencies = run_simulation(init_G, c, model='linear', graph_type='directed')
            eff = calculate_efficiency(G_eq, c, 'linear', delta=0, graph_type='directed')
            out_lines.append(f"  Init: {init_name:<10} | Steps: {len(history)-1:<4} | Efficiency: {eff:<6.2f} | Num Edges: {G_eq.number_of_edges()}")
            plot_graph(G_eq, f"Directed Linear, {label}, Init: {init_name}", f"directed_linear_{label}_{init_name}.png")
            plot_convergence(efficiencies, f"Efficiency: Directed Linear, {label}, Init: {init_name}", f"conv_directed_linear_{label}_{init_name}.png")

    # 3. Undirected Connections
    out_lines.append("\\n=== Part 3: Undirected Connections ===")
    delta = 0.5
    # thresholds: delta - delta**2 = 0.25
    # delta + (n/2-1)*delta**2 = 0.5 + (3.5)*0.25 = 1.375
    c_vals_conn = [0.1, 0.8, 2.0]
    c_labels_conn = ['c_small', 'c_medium', 'c_large']
    for c, label in zip(c_vals_conn, c_labels_conn):
        out_lines.append(f"\\nRegime: {label} (c = {c})")
        inits = generate_inits(n, 'undirected')
        for init_name, init_G in inits.items():
            G_eq, history, efficiencies = run_simulation(init_G, c, model='connections', delta=delta, graph_type='undirected')
            eff = calculate_efficiency(G_eq, c, 'connections', delta=delta, graph_type='undirected')
            out_lines.append(f"  Init: {init_name:<10} | Steps: {len(history)-1:<4} | Efficiency: {eff:<6.2f} | Num Edges: {G_eq.number_of_edges()}")
            plot_graph(G_eq, f"Undirected Connections, {label}, Init: {init_name}", f"undirected_conn_{label}_{init_name}.png")
            plot_convergence(efficiencies, f"Efficiency: Undirected Connections, {label}, Init: {init_name}", f"conv_undirected_conn_{label}_{init_name}.png")

    with open('/Users/mhanisch/projects/psets/q2/144/hw6/code/simulate_networks.py.out', 'w') as f:
        f.write("\\n".join(out_lines))
```

solve_maze.py

```python
import sys
import random
import matplotlib.pyplot as plt
import numpy as np
from maze import Maze

def get_next_state_intended(state, direction):
    # Action map from maze: 0=N, 1=S, 2=E, 3=W
    # but wait, the maze has moves explicitly defined:
    moves = [(0,1), (0, -1), (1,0), (-1, 0)]
    x_change = moves[direction][0]
    y_change = moves[direction][1]

    x, y = state
    return ((x+x_change) % 10, (y+y_change) % 10)

def train_agent():
    env = Maze()

    # Parameters
    alpha = 0.5
    epsilon_start = 1.0
    epsilon_min = 0.05
    epsilon_decay = 0.995
    epochs = 998

    # Rewards / Valuations
    R_unexplored = 0.0 # Standard plain room
    R_snake = -100.0
    R_exit = 100.0

    # Initialize V table
    V = {}
    for x in range(10):
        for y in range(10):
            V[(x, y)] = R_unexplored

    # Hardcode terminal states (so they never decay from their terminal value)
    for snake in env.snakes:
        V[snake] = R_snake
    for exit in env.escapes:
        V[exit] = R_exit

    epsilon = epsilon_start

    training_steps = []

    for epoch in range(epochs):
        state = env.reset()
        t = 0
        while True:
            # Epsilon greedy policy based on strictly adjacent rooms
            if random.random() < epsilon:
                action = random.randint(0, 3)
            else:
                best_val = -float('inf')
                best_actions = []
                for a in range(4):
                    next_s = get_next_state_intended(state, a)
                    val = V[next_s]
                    if val > best_val:
                        best_val = val
                        best_actions = [a]
                    elif val == best_val:
                        best_actions.append(a)
                action = random.choice(best_actions)

            next_state, reward_type = env.step(action)

            # The environment step determines our actual next state.
            # Update rule from the problem: V(i) = (1-a)V(i) + a * V(i')
            if state not in env.snakes and state not in env.escapes:
                V[state] = (1 - alpha) * V[state] + alpha * V[next_state]
```

```
            state = next_state
            t += 1

            if reward_type == 1 or reward_type == -1:
                break

        epsilon = max(epsilon_min, epsilon * epsilon_decay)
        training_steps.append(t)

    return V, training_steps, env

def test_agent(V, env, episodes=1000):
    success_count = 0
    test_steps = []
    for _ in range(episodes):
        state = env.reset()
        t = 0
        success = False
        while True:
            # Greedy
            best_val = -float('inf')
            best_actions = []
            for a in range(4):
                next_s = get_next_state_intended(state, a)
                val = V[next_s]
                if val > best_val:
                    best_val = val
                    best_actions = [a]
                elif val == best_val:
                    best_actions.append(a)
            action = random.choice(best_actions)

            next_state, reward_type = env.step(action)
            state = next_state
            t += 1

            if reward_type == 1:
                success_count += 1
                success = True
                break
            elif reward_type == -1:
                break

            # Prevent infinite loops in testing
            if t > 500:
                break
        test_steps.append(t)

    return success_count, test_steps

def plot_valuations(V, env):
    grid = np.zeros((10, 10))
    for x in range(10):
        for y in range(10):
            grid[y, x] = V[(x, y)] # plotting y on rows, x on cols matches typical Cartesian grid if
    inverted, but let's just use standard matrix

    plt.figure(figsize=(10, 8))
    # By passing origin='upper', row 0 is at Top.
    # Col x, Row y matches array index grid[y, x]
    im = plt.imshow(grid, cmap="RdYlGn")
    plt.colorbar(im)
    plt.title("Learned Room Valuations V(s)")
    plt.xlabel("X Coordinate")
    plt.ylabel("Y Coordinate")

    # Annotate numbers
    for x in range(10):
        for y in range(10):
            val = grid[y, x]
            # Print integer part so "99.9" becomes "99" instead of rounding to "100.0"
            plt.text(x, y, f"{int(val)}", ha='center', va='center', color='black' if -50 < val < 50 else '
    white', fontsize=10)

    # Annotate snakes and escapes
```

```
    for (x, y) in env.snakes:
        plt.text(x, y, 'S', ha='center', va='center', color='black', fontsize=16, fontweight='bold')
    for (x, y) in env.escapes:
        plt.text(x, y, 'E', ha='center', va='center', color='blue', fontsize=16, fontweight='bold')

    plt.text(0, 0, 'START', ha='center', va='center', color='purple', fontsize=12, fontweight='bold')

    plt.tight_layout()
    plt.savefig('../latex/figs/maze_learned_values.png')
    plt.close()

if __name__ == "__main__":
    V, t_steps, env = train_agent()
    print("Training finished. Running test episodes...")
    successes, test_steps = test_agent(V, env, 1000)
    print(f"Successes: {successes}/1000")
    if successes == 1000:
        print("Agent successfully escapes 1000/1000 times!")

    plot_valuations(V, env)
    print("Saved valuation grid to latex/figs/maze_learned_values.png")
```